МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра электронных приборов и устройств

ОТЧЕТ

по научно-исследовательской работе

Тема: Разработка алгоритма сшивки рентгеновских изображений, полученных с помощью рентгенографической системы для неразрушающего контроля печатных плат

Студент гр. 0202 _____ Кудрявцев А.С.

Руководитель _____ Ларионов И.А.

Санкт-Петербург
2024

ЗАДАНИЕ

НА НАУЧНО-ИССЛЕДОВАТЕЛЬСКУЮ РАБОТУ

Студент Кудрявцев А.С.

Группа 0202

Тема НИР: Разработка алгоритма сшивки рентгеновских изображений, полученных с помощью рентгенографической системы для неразрушающего контроля печатных плат

Задание на НИР:

Разработать алгоритм, позволяющий осуществлять сшивку изображений, получаемых с рентгенографической системы для неразрушающего контроля печатных плат

Сроки выполнения НИР: 14.09.2024 – 20.12.2024

Дата сдачи отчета: 24.12.2024

Дата защиты отчета: 24.12.2024

Студент _____ Кудрявцев А.С.

Руководитель _____ Ларионов И.А.

АННОТАЦИЯ

В данной работе производится описание современных средств обработки изображений для осуществления сшивки получаемых рентгенограмм для получения целостной картины исследуемого объекта. Разрабатывается алгоритм, позволяющий производить автоматическую, корректную сшивку рентгенографических снимков печатных плат, для обеспечения качественного неразрушающего контроля в производстве электроники.

SUMMARY

This paper describes modern image processing tools for stitching the resulting radiographs to obtain a complete picture of the object being studied. An algorithm is being developed that allows for automatic, correct stitching of radiographic images of printed circuit boards to ensure high-quality non-destructive testing in electronics production.

СОДЕРЖАНИЕ

1. Цифровая обработка изображений 6
1.1. Особенности цифрового рентгеновского изображения 6
1.2. Методы сшивки изображений. Выделение ключевых точек изображения 7
1.3. Аффинные и гомографические преобразования изображения 12
2. Разработка и реализация алгоритма сшивания рентгеновских изображений 15
2.1. Разработка алгоритма сшивки рентгеновских изображений 15
2.2. Выбор среды, языка программирования и библиотек для создания алгоритма 15
2.3. Предобработка изображений 16
2.4. Вычисление и сопоставление ключевых точек изображений 17
2.5. Аффинное преобразование изображения и сшивка 19
3. Тестирование алгоритма сшивки рентгеновских изображений 21
3.1. Томографическая система используемая при тестах 21
3.2. Набор тестовых данных 23
3.3. Результаты тестирования 23
Список использованных источников 28
Приложение А 29

ВВЕДЕНИЕ

В современных условиях развития электронной промышленности возрастает потребность в эффективных методах контроля качества печатных плат, которые являются ключевыми компонентами большинства электронных устройств. Одним из наиболее распространённых и надёжных методов анализа структуры таких плат является использование рентгеновских томографических систем, позволяющей выявлять дефекты, скрытые под слоями проводников и материалов.

Сшивка рентгеновских изображений представляет собой важный этап в процессе контроля, поскольку размеры плат зачастую превышают поле зрения рентгеновской камеры. Для обеспечения точности анализа необходимо объединение множества снимков в единое изображение с минимальными погрешностями. Однако существует множество трудностей с обеспечением достаточной точности и скорости обработки, особенно при наличии сложных текстур или недостаточной информации на отдельных участках снимков.

Целью данной работы является разработка алгоритма сшивки рентгеновских изображений, обеспечивающего высокую точность, устойчивость к шумам и эффективность в условиях ограниченных вычислительных ресурсов. Для достижения этой цели в работе рассматриваются существующие методы обработки изображений и оценивается их эффективность на реальных данных неразрушающего контроля печатных плат.

1. ЦИФРОВАЯ ОБРАБОТКА ИЗОБРАЖЕНИЙ

1.1. Особенности цифрового рентгеновского изображения

Изображение представляет из себя образ, в той или иной степени подобный изображаемому объекту.

Оптическое изображение – это картина, получаемая в результате прохождения через оптическую систему отраженных от объекта световых лучей и воспроизводящая его контуры и детали на некоторой поверхности.

Цифровое изображение - массив данных, полученный путем дискретизации (аналого-цифрового преобразования) оригинала. Будучи закодированным с помощью особого алгоритма и записанным на носитель, этот массив данных становится файлом.

Получение рентгеновского снимка при использовании рентгеновского аппарата серии «…» производится с помощью цифрового рентгеновского детектора. На выходе получается 16-ти битное изображение в градациях серого, где уровень яркости варьируется от 0 до 65535. Но при этом при просмотре такого изображения монитор может отображать только 256 градаций серого при обработке необходимо преобразование в 8-битный формат.

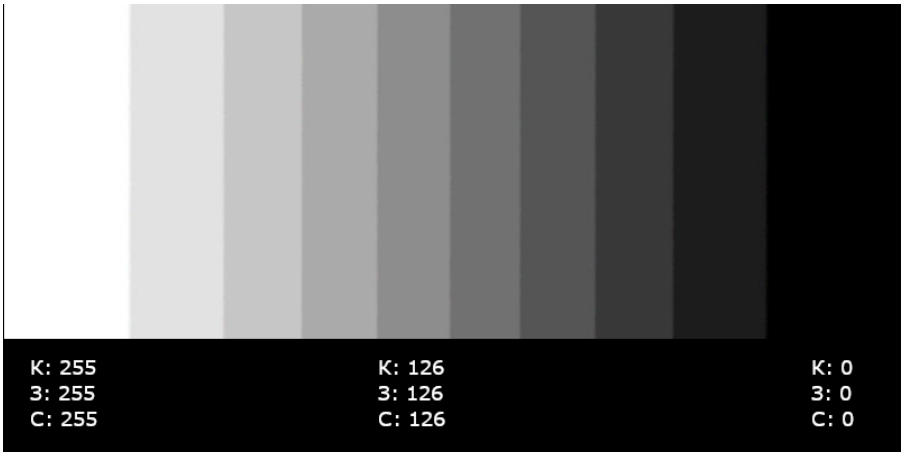


Рисунок 1.1 – 8-битовая шкала градаций серого

При таком преобразовании возможны потери, а следовательно, и ухудшение видимости дефектов, которые попали на более темную или более светлую часть изображения. Таким образом, при исследовании определенных

участков необходимо устанавливать тот интервал яркостей, который будет наиболее значимым для отображения конкретного снимка. Также возможна ручная установка границ белого и черного или использование математических алгоритмов, позволяющих разбить рентгеновское изображение на области и производить «сжатие» и «растяжение» требуемого динамического диапазона в зависимости от поставленных задач.

1.2. Методы сшивки изображений. Выделение ключевых точек изображения

Сшивка рентгеновских изображений — это процесс объединения нескольких снимков, сделанных с перекрытием, в единое изображение.

Сначала изображения проходят предварительную обработку. На этом этапе нормализуется яркость, чтобы устранить неравномерность освещения, повышается контрастность с использованием методов, таких как адаптивное выравнивание гистограмм (CLAHE), а также удаляется шум с помощью фильтров, например, медианного или гауссового. Это помогает улучшить качество исходных данных перед дальнейшими шагами.

Следующим ключевым этапом является поиск точек совпадения. Для этого применяются алгоритмы, такие как SIFT, SURF или ORB, которые находят устойчивые ключевые точки. Эти точки описываются с помощью дескрипторов, представляющих локальное окружение каждой из них. Затем осуществляется сопоставление дескрипторов между изображениями с использованием методов поиска ближайших соседей, таких как k-NN или FLANN. Ложные совпадения отсекаются с помощью алгоритма RANSAC, который также помогает определить геометрическую трансформацию между изображениями.

После нахождения точек совпадения вычисляется преобразование, необходимое для совмещения изображений. Тип трансформации зависит от характера данных: для рентгеновских изображений часто используется аффинная трансформация или проективная (гомография), чтобы учитывать искажения. Матрица трансформации строится на основе точек совпадения, а затем одно изображение преобразуется относительно другого. Для этого

применяются методы интерполяции, такие как билинейная или бикубическая, чтобы избежать искажения деталей.

На этапе слияния изображений необходимо устранить видимые переходы в зонах перекрытия. Это достигается корректировкой яркости и интенсивности пикселей в области шва. Для плавного объединения могут использоваться методы, такие как взвешенное усреднение или многополосное разложение (multi-band blending), которые помогают минимизировать видимость границ между изображениями.

После сшивки результат оценивается. Это может быть визуальная проверка на отсутствие швов и искажений, а также использование метрик качества, таких как индекс структурного сходства (SSIM) или взаимная информация, чтобы количественно оценить качество объединения. Особенности рентгеновских изображений, такие как слабая текстурированность и наличие шума, могут усложнить процесс, требуя дополнительных шагов обработки или настройки алгоритмов. Однако правильно настроенные методы позволяют создавать единые изображения высокого качества, пригодные для анализа [4].

Остановимся подробнее на методе выделения ключевых точек SIFT для сшивки рентгеновских изображений:

Алгоритм выделения ключевых точек SIFT

SIFT (Scale-Invariant Feature Transform) — это алгоритм, предназначенный для извлечения устойчивых локальных признаков изображения, которые инвариантны к масштабным изменениям, повороту, и относительно устойчивы к изменению освещения и шумам. Разработанный Дэвидом Лоу в 2004 году, SIFT состоит из нескольких ключевых этапов, каждый из которых имеет строгое математическое обоснование [5].

1) Построение масштабно-пространственной пирамиды (Scale-Space Extrema Detection)

SIFT работает с изображениями в разных масштабах для поиска ключевых точек, инвариантных к изменению масштаба.

Первым делом происходит для изображения *I(x,y)* строится семейство изображений *L(x,y,σ)*, где каждое из них получается путём свёртки исходного изображения с гауссовым ядром *G(x,y,σ)*:

$$L(x, y, \sigma) = G(x, y, \sigma) * I(x, y)$$

где $G(x, y, \sigma)$ — гауссово ядро:

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma 2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Далее происходит вычисление разностных изображений Гаусса (DoG). Они вычисляются как разность двух соседних изображений, размытых с разными параметрами *σ*:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

Как показано на рисунке 1, исходное изображение последовательно свёртывается с гауссовыми ядрами для получения изображений, разделённых постоянным фактором k в пространстве масштаба (левый столбец на рисунке). Ошибка приближения будет равна нулю, когда k = 1. Для каждой октавы масштаба (умножение σ) верхнее изображение повторно подбирается, выбирая каждый второй пиксель в каждой строке и столбце, чтобы создать начальное изображение для следующей октавы с удвоенным σ, что значительно упрощает вычисления. Смежные изображения в стеке октавы вычитаются для получения изображений DoG, как показано на рисунке 1.

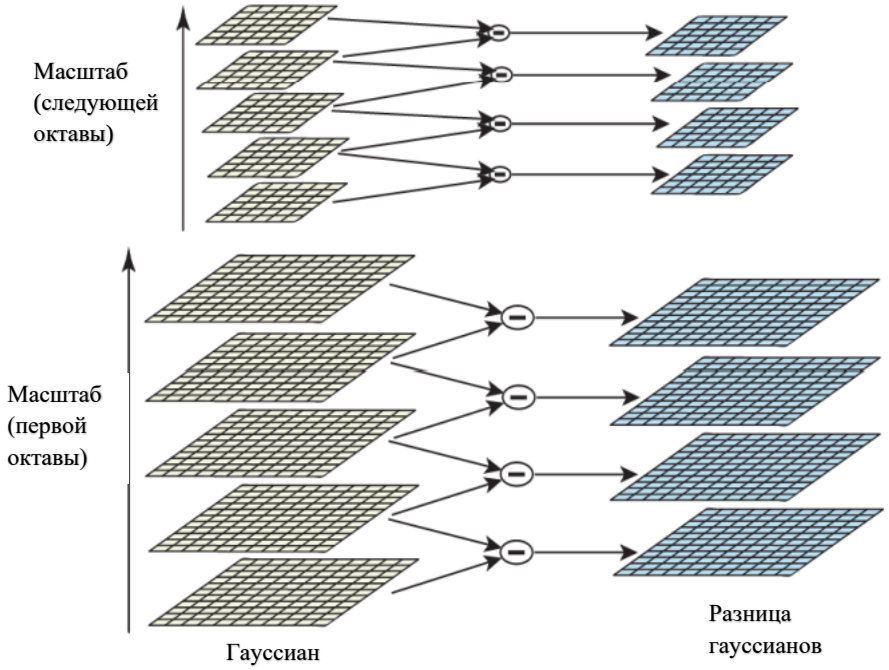


Рисунок 1.1 – Построение разностных изображений Гаусса [1]

Далее происходит вычисление экстремумов (максимумов или минимумов), которые определяются путём сравнения каждой точки в пространстве *(x,y)* и масштабе σ с 26 соседними точками.

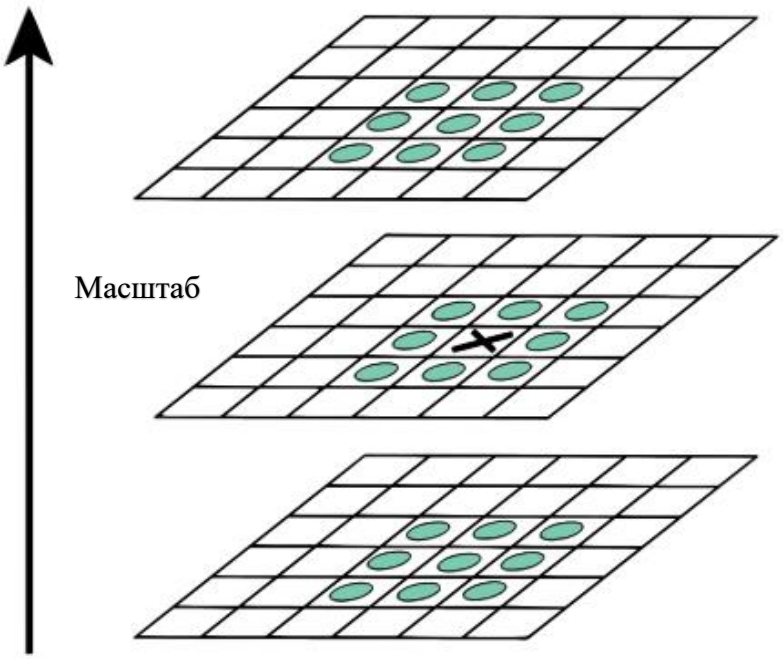


Рисунок 1.2 – Распознавание максимумов и минимумов изображений DoG

2) Точная локализация ключевых точек

Этот шаг выполняет детальную подгонку к близлежащим данным для местоположения, масштаба и соотношения главных кривизн, чтобы мы могли удалить точки с низким контрастом или плохо локализованные вдоль края.
Для повышения точности экстремумы уточняются с помощью аппроксимации.

Аппроксимация функции $D(x,y,\sigma)$ выполняется с использованием разложения в ряд Тейлора [1]:

$$D^x = D + \frac{\partial D^T}{\partial x}x + \frac{1}{2}x^T\frac{\partial^2 D}{\partial x^2}x$$

Здесь x — смещение экстремума. Решая уравнение:

$$\frac{\partial \widehat{D}}{\partial x} = 0$$

находится уточнённое положение экстремума. Если контраст точки слишком низкий или она находится на краю объекта (определяется по соотношению собственных значений матрицы Гессиана), точка отбрасывается.

3) Определение ориентации ключевых точек

Чтобы обеспечить инвариантность к повороту, каждой ключевой точке присваивается ориентация.

Для начала производится вычисление градиентов вокруг ключевой точки:

$$m(x,y) = \sqrt{\big(L(x+1,y) - L(x-1,y)\big)2 + \big(L(x,y+1) - L(x,y-1)\big)},$$

$$\theta(x,y) = \arctan\left(\frac{L(x+1,y) - L(x-1,y)}{L(x,y+1) - L(x,y-1)}\right),$$

где $m(x,y)$ – значение, а $\theta(x,y)$ - направление градиента.

В области вокруг точки строится гистограмма ориентаций, где высота каждого бина пропорциональна сумме величин градиентов. Основная ориентация определяется как максимальный бином гистограммы.

4) Формирование дескрипторов ключевых точек

Для каждой ключевой точки создаётся дескриптор, описывающий её локальное окружение. Он является инвариантным к изменению освещения или трехмерной точки обзора. Этого можно достичь, выбрав окно вокруг ключевой

точки и сгенерировав векторы признаков. Предположим, мы выбрали окно 16x16, затем мы разбиваем его на окна 4x4, как показано на рисунке 3. Для каждого окна мы вычисляем величину градиента и ориентацию в каждой точке.

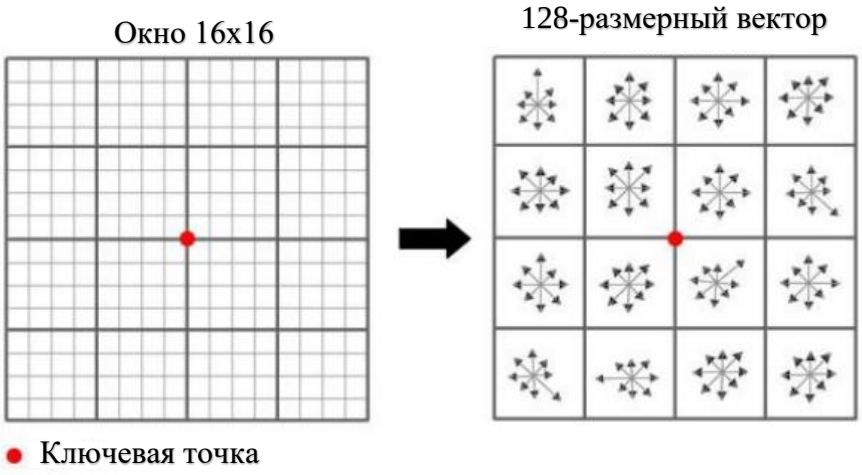


Рисунок 1.3 – Создание дескриптора ключевой точки с помощью градиентов изображения [3]

Опять же, величина взвешивается в соответствии с расстоянием от ключевой точки (используется функция взвешивания Гаусса), т. е. градиенты, которые далеко от ключевой точки, будут добавлять меньшие значения. Таким образом, гистограмма ориентации с 8 ячейками несет 8 признаков для каждого небольшого окна. Таким образом, для окна 16x16 мы можем иметь вектор признаков ключевой точки с 16x8=128 признаками в нем, который уникально описывает ключевую точку.

1.3. Аффинные и гомографические преобразования изображения

Аффинное отображение — это отображение, при котором параллельные прямые переходят в параллельные прямые, пересекающиеся в пересекающиеся, скрещивающиеся в скрещивающиеся; сохраняются отношения длин отрезков, лежащих на одной прямой (или на параллельных прямых), и отношения площадей фигур.



Рисунок 1.4 - Конформные преобразования, верхний ряд слева направо: исходное изображение, сдвиг, отражение, вращение; нижний ряд: однородное масштабирование [2].

Базовыми преобразованиями являются конформные преобразования, скос и неоднородное масштабирование. Произвольное аффинное преобразование можно получить при помощи последовательного произведения матриц базовых преобразований. В непрерывной геометрии любое аффинное преобразование имеет обратное аффинное преобразование, а произведение прямого и обратного дает единичное преобразование, которое оставляет все точки на месте. Аффинные преобразования являются подмножеством проекционных преобразований.

Проекционное отображение (гомографическое) — это отображение, при котором прямые линии остаются прямыми линиями, однако геометрия фигуры может быть нарушена, т.к. данное отображение в общем случае не сохраняет параллельности линий. Свойством, сохраняющимся при проективном преобразовании, является коллинеарность точек: три точки, лежащие на одной прямой (коллинеарные), после преобразования остаются на одной прямой. Проекционное отображение может быть как параллельным (изменяется масштаб), так и проективным (изменяется геометрия фигуры).

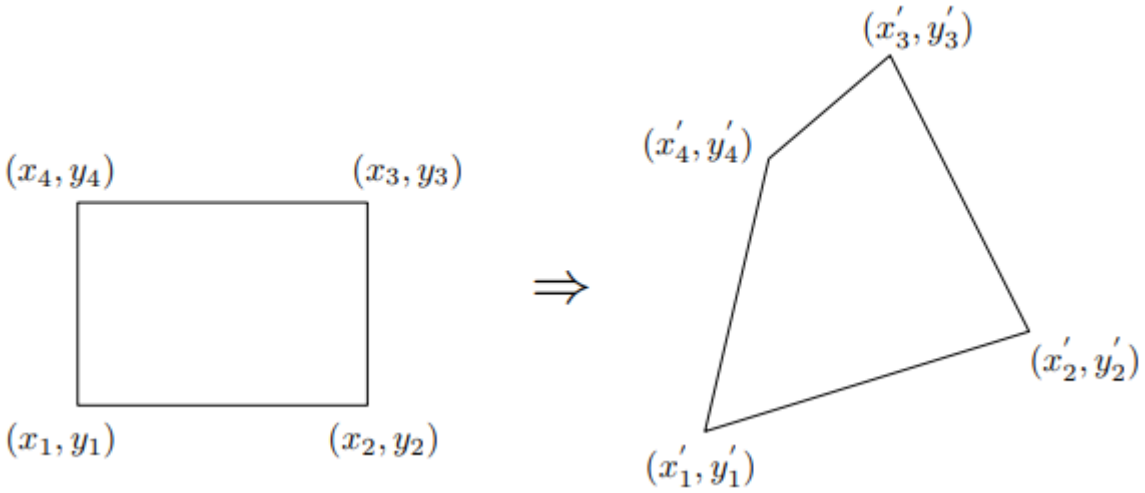


Рисунок 1.5 – Проекционное проективное отображение: прямые остались прямыми, но параллельные отобразились в скрещивающиеся.

2. РАЗРАБОТКА И РЕАЛИЗАЦИЯ АЛГОРИТМА СШИВАНИЯ РЕНТГЕНОВСКИХ ИЗОБРАЖЕНИЙ

2.1. Разработка алгоритма сшивки рентгеновских изображений

Для качественной сшивки рентгеновских изображений необходимо получить изображения с рентгеновского детектора, произвести их обработку для удаления шумов и нормализации гистограммы изображения, произвести вычисление ключевых точек и произвести сопоставление этих точек между двумя изображениями и сшить изображения при этом смешивая их области пересечения. Блок схема алгоритма сшивки двух изображений представлена на рисунке 2.1.

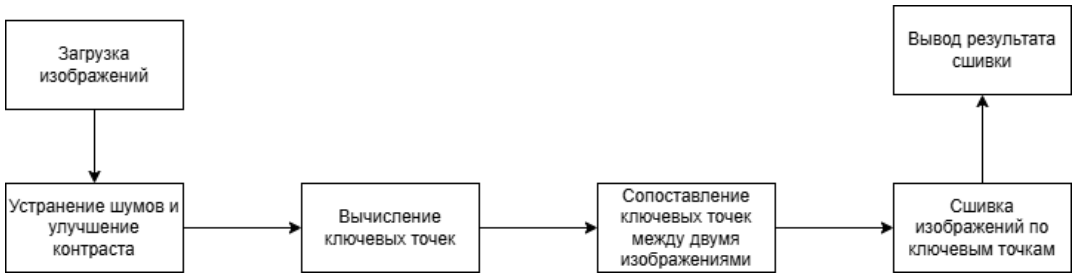


Рисунок 2.1 – Блок схема алгоритма сшивки двух изображений

2.2. Выбор среды, языка программирования и библиотек для создания алгоритма

Для реализации алгоритма был выбран язык программирования C++ и библиотека методов компьютерного зрения OpenCV. Средой разработки стало ПО «VS Code», позволяющее писать код на разных языках, с подключением множества библиотек и свободным выбором компилятора. Подключение используемых библиотек представлено в листинге 2.1.

Библиотека «OpenCV» (Open Source Computer Vision Library) используется для решения широкого спектра задач компьютерного зрения и обработки изображений. Она предоставляет инструменты для работы с изображениями, видео, графикой, а также алгоритмы машинного обучения. OpenCV широко применяется в научных исследованиях, промышленности, робототехнике, медицине и других областях.

Библиотека «chrono» в C++ — это часть стандартной библиотеки <chrono>, которая предоставляет функциональность для работы со временем и датами. Она

используется для измерения временных интервалов, работы с временными метками, управления задержками и выполнения других задач, связанных со временем.

Библиотека <string> предоставляет класс std::string, который используется для работы со строками. Это более мощный и удобный инструмент по сравнению с традиционными С-строками (массивами символов).

Библиотека <iostream> предоставляет базовые средства для ввода и вывода данных. Она включает классы std::cin, std::cout, std::cerr, которые используются для работы с консольным вводом и выводом.

Листинг 2.1 – Подключение используемых библиотек

```
#include <iostream>
#include <string>
#include "opencv2/opencv.hpp"
#include <opencv2/features2d.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/calib3d.hpp>
#include <chrono>
```

2.3. Предобработка изображений

Для качественного нахождения ключевых точек необходимо устранить возможные шумы и недостатки сшиваемых изображений. Поскольку рентгеновские изображения могут получиться недостаточно контрастными необходимо произвести динамическое растяжение гистограммы для равномерного распределения яркостей пикселей.

Для уменьшения шумов на изображении используется функция GaussianBlur(input_img, output_img, kernel_size, sigma), где input_img – исходное изображение, output_img – итоговое изображение, kernel_size – размер ядра, sigma – стандартное отклонение.

Для растяжения изображения по динамическому диапазону испольуется функция normalize(image, normalize_image, 0, 255, cv::NORM_MINMAX), где image – исходное изображение, normalize_image – итоговое изображение, 0 и

255 – это границы диапазона, NORM_MINMAX – флаг, устанавливающий для максимумов и минимумов значений яркостей пикселей.

Для проведения эквализации гистограммы изображения используется функция cv::equalizeHist(image, equalized_image). Она производит равномерную нормализацию гистограммы рентгеновского изображения, повышая его контрастность.

В листинге 2.2 приводится код с проведением предварительной обработки изображений.

Листинг 2.2 – Предварительная обработка изображений

```
cv::GaussianBlur(img1, img1, cv::Size(7,7), 1.5);
cv::GaussianBlur(img2, img2, cv::Size(7,7), 1.5);
cv::normalize(img1, img1, 0, 255, cv::NORM_MINMAX);
cv::normalize(img1, img1, 0, 255, cv::NORM_MINMAX);
cv::equalizeHist(img1,img1);
cv::equalizeHist(img2,img2);
```

2.4. Вычисление и сопоставление ключевых точек изображений

Для вычисления ключевых точек используется алгоритм SIFT, как алгоритм, показывающий наибольшую эффективность и точность.

В листинге 2.3 приведен код, производящий вычисление ключевых точек и дескрипторов двух изображений

Листинг 2.3 – Вычисление ключевых точек и дескрипторов

```
std::vector<cv::KeyPoint> keypoints1, keypoints2;
cv:: Mat descriptors1, descriptors2;
cv::Ptr<cv::SIFT> sift =
cv::SIFT::create();//cv::AKAZE::DESCRIPTOR_KAZE, 3, 0.001f,
4);
std:: cout << "checkpointSIFT1" << std::endl;
sift->detectAndCompute(img1, cv::Mat(), keypoints1,
descriptors1);
std:: cout << "checkpointSIFT2" << std::endl;
sift->detectAndCompute(img2, cv::Mat(), keypoints2,
descriptors2);
```

Сопоставление дескрипторов двух изображений производится с помощью функции FlannBasedMatcher(new cv::flann::KDTreeIndexParams(), new cv::flann::SearchParams()). Данный алгоритм является эффективным для поиска ближайших соседей и сопоставления дескрипторов ключевых точек в больших наборах данных.

В листинге 2.4 приведен код, производящий сопоставление дескрипторов двух изображений

Листинг 2.4 – Вычисление ключевых точек и дескрипторов

```
cv::FlannBasedMatcher
matcher(new cv::flann::KDTreeIndexParams(),
new cv::flann::SearchParams());
descriptors1.convertTo(descriptors1, CV_32F);
descriptors2.convertTo(descriptors2, CV_32F);
std::vector<std::vector<cv::DMatch>> knn_matches;
matcher.knnMatch(descriptors1, descriptors2, knn_matches, 2);
```

На рисунке 2.2 изображен пример сопоставления дескрипторов и ключевых точек двух изображений.

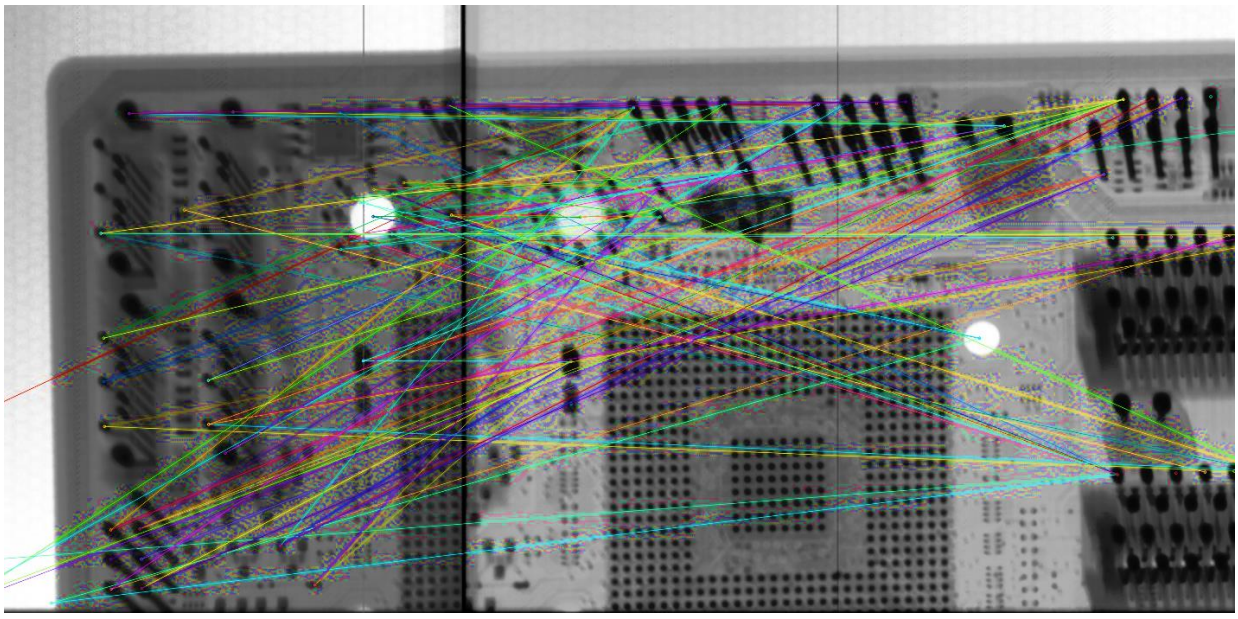


Рисунок 2.2 – Сопоставление ключевых точек изображений

2.5. Аффинное преобразование изображения и сшивка

Для обеспечения точности при наложении изображений и компенсации различных искажений и движений необходимо произвести аффинное преобразование. В листинге 2.5 приведен код позволяющий преобразовать два изображения.

Листинг 2.5 – Аффинное преобразование сшиваемых изображений

```
cv::Mat affine_matrix = cv::estimateAffinePartial2D(points2,
points1);
std::vector<cv::Point2f> img2_corners(4);
img2_corners[0] = cv::Point2f(0,0);
img2_corners[1] = cv::Point2f(static_cast<float>(img2.cols),
0);
img2_corners[2] =
cv::Point2f(0, static_cast<float>(img2.rows));
img2_corners[3] = cv::Point2f(static_cast<float>(img2.cols),
static_cast<float>(img2.rows));
std::vector<cv::Point2f> img2_transformed_corners(4);
cv::transform(img2_corners, img2_transformed_corners,
affine_matrix);

float min_x = std::min(0.0f,
std::min({img2_transformed_corners[0].x,
img2_transformed_corners[1].x, img2_transformed_corners[2].x
}));

float min_y = std::min(0.0f,
std::min({img2_transformed_corners[0].y,
img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));

float max_x = std::max(static_cast<float>(img1.cols),
std::max({img2_transformed_corners[0].x,
img2_transformed_corners[1].x,
img2_transformed_corners[2].x}));
```

```
float max_y = std::max(static_cast<float>(img1.rows),
std::max({img2_transformed_corners[0].y,
img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));

cv::Size result_size(img1.cols,
static_cast<int>(std::ceil(max_y - min_y)));
cv::Mat result(result_size, img1.type(),
cv::Scalar::all(255));
cv:: Mat translation_matrix = (cv::Mat_<double>(2, 3) << 1,
0, -min_x, 0 , 1, -min_y);
cv::warpAffine(img1, result, translation_matrix, result_size,
cv::INTER_LINEAR, cv::BORDER_REFLECT);
cv::Mat img2_transformed;
cv::Mat img2_shifted;
cv::warpAffine(img2, img2_shifted, translation_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REFLECT);
cv::warpAffine(img2_shifted, img2_transformed, affine_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REFLECT);
```

Следующим шагом является сшивание изображений путем их плавного смешивания, для уменьшения заметности перехода. Данный метод сопоставления представлен в листинге 2.6.

Листинг 2.6 – Плавное смешивание изображений

```
cv::Mat blended;
blended = result.mul(1.0 - mask) +
img2_transformed.mul(mask);
```

3. ТЕСТИРОВАНИЕ АЛГОРИТМА СШИВКИ РЕНТГЕНОВСКИХ ИЗОБРАЖЕНИЙ

3.1. Томографическая система используемая при тестах

Для получения рентгеновских изображений печатных плат используется разрабатываемая предприятием ЗАО «ЭЛТЕХ-Мед» рентгенографическая система для неразрушающего контроля печатных плат «ПРДУ-Электроник». Данная система представлена на рисунке 3.1.



Рисунок 3.1 – Рентгенографическая система для неразрушающего контроля печатных плат «ПРДУ-Электроник»

Местом расположения исследуемых объектов является предметный столик, изображенный на рисунке 3.2. Данный столик имеет трехосную систему перемещений, что позволяет точно подстраивать исследуемый объект и получать необходимое увеличение на снимке.

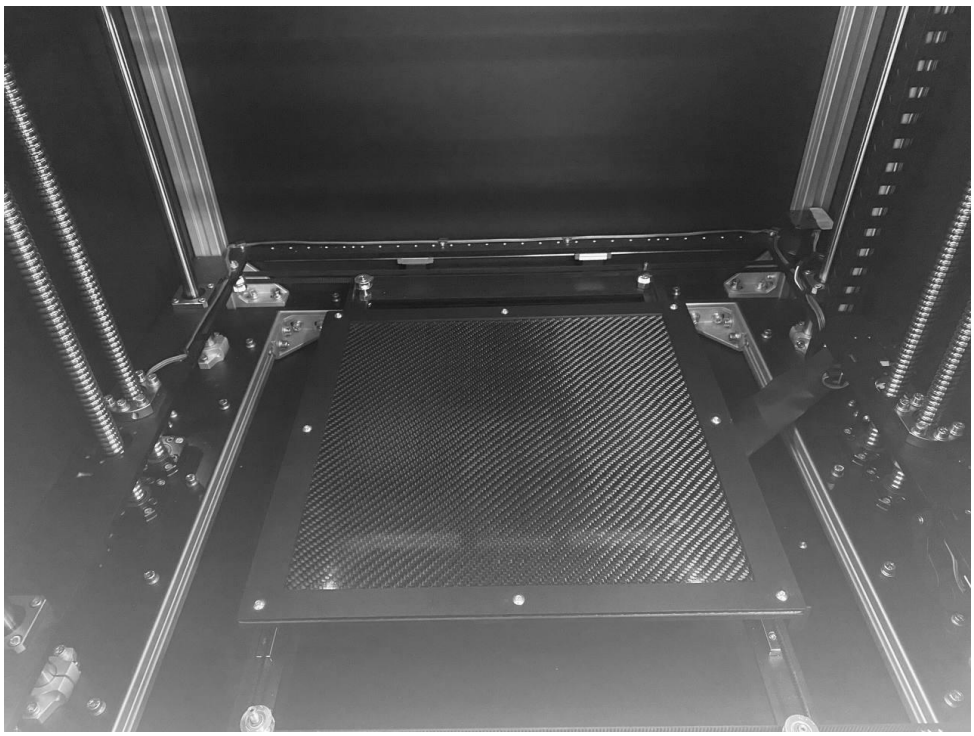


Рисунок 3.2 – Предметный столик рентгенографической системы

Считывание рентгеновского изображения и получение цифрового изображения производится цифровым приемником, изображенным на рисунке 3.3.

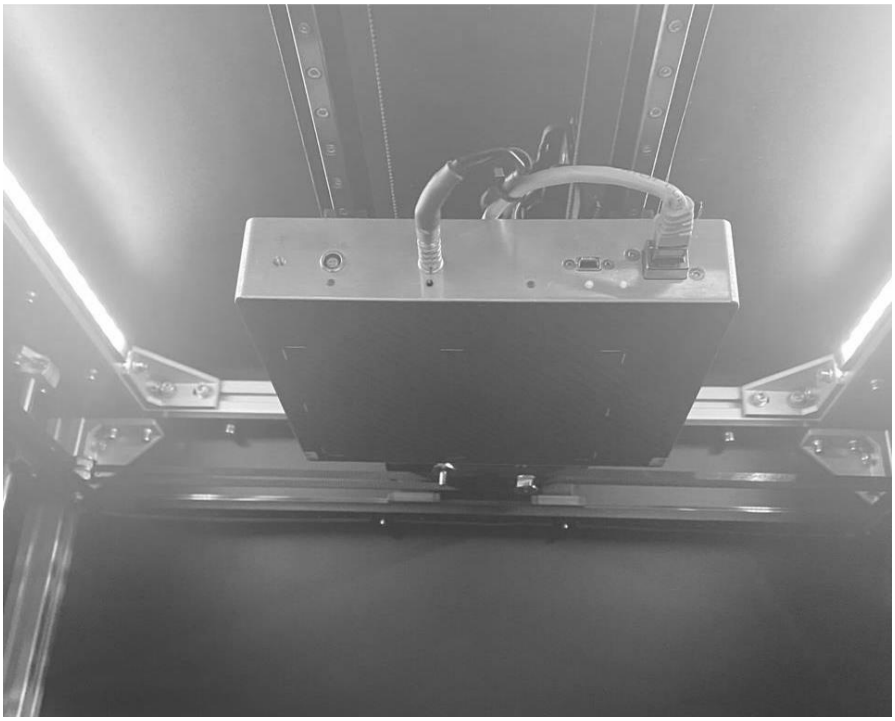


Рисунок 3.3 – Цифровой приемник рентгенографической системы

3.2. Набор тестовых данных

В качестве набора тестовых данных используются 16-битные рентгеновские изображения печатной платы, снятые с разным коэффициентом геометрического увеличения и напряжением на рентгеновском аппарате. Таким образом, получено 4 комплекта изображений, включающих в себя по 20 изображений, являющихся частями одного целого. Один из данных комплектов представлен на рисунке 3.4.

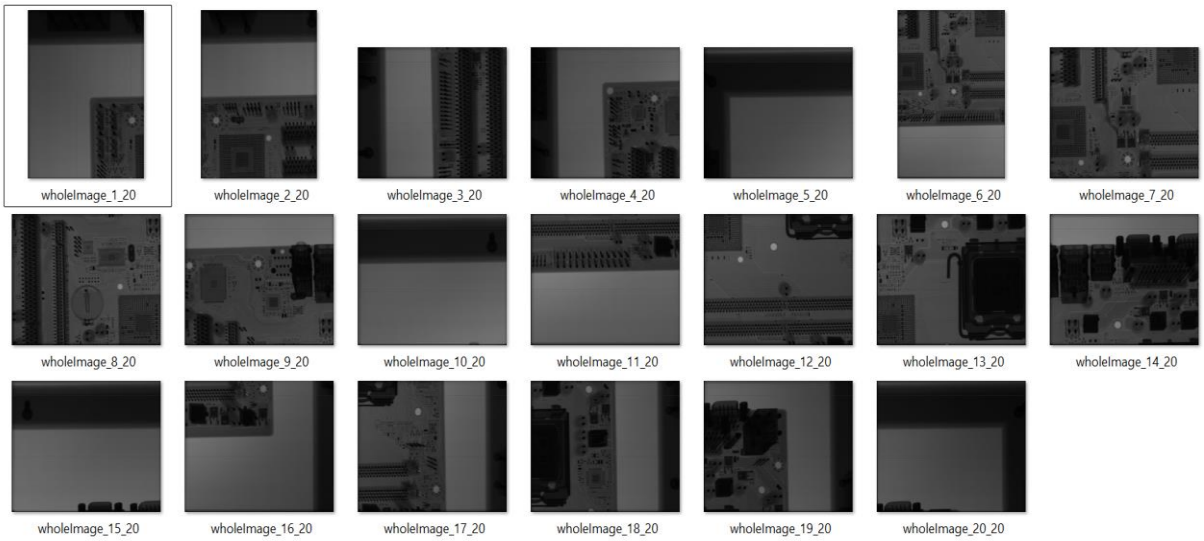


Рисунок 3.4 – Тестовый комплект изображений, полученных с детектора

3.3. Результаты тестирования

В результате работы алгоритма при обработке первого комплекта изображений, получено целостное изображение печатной платы в высоком разрешении, что позволяет наблюдать общую целостную картину исследуемого объекта. Полученное изображение приведено на рисунке 3.5.

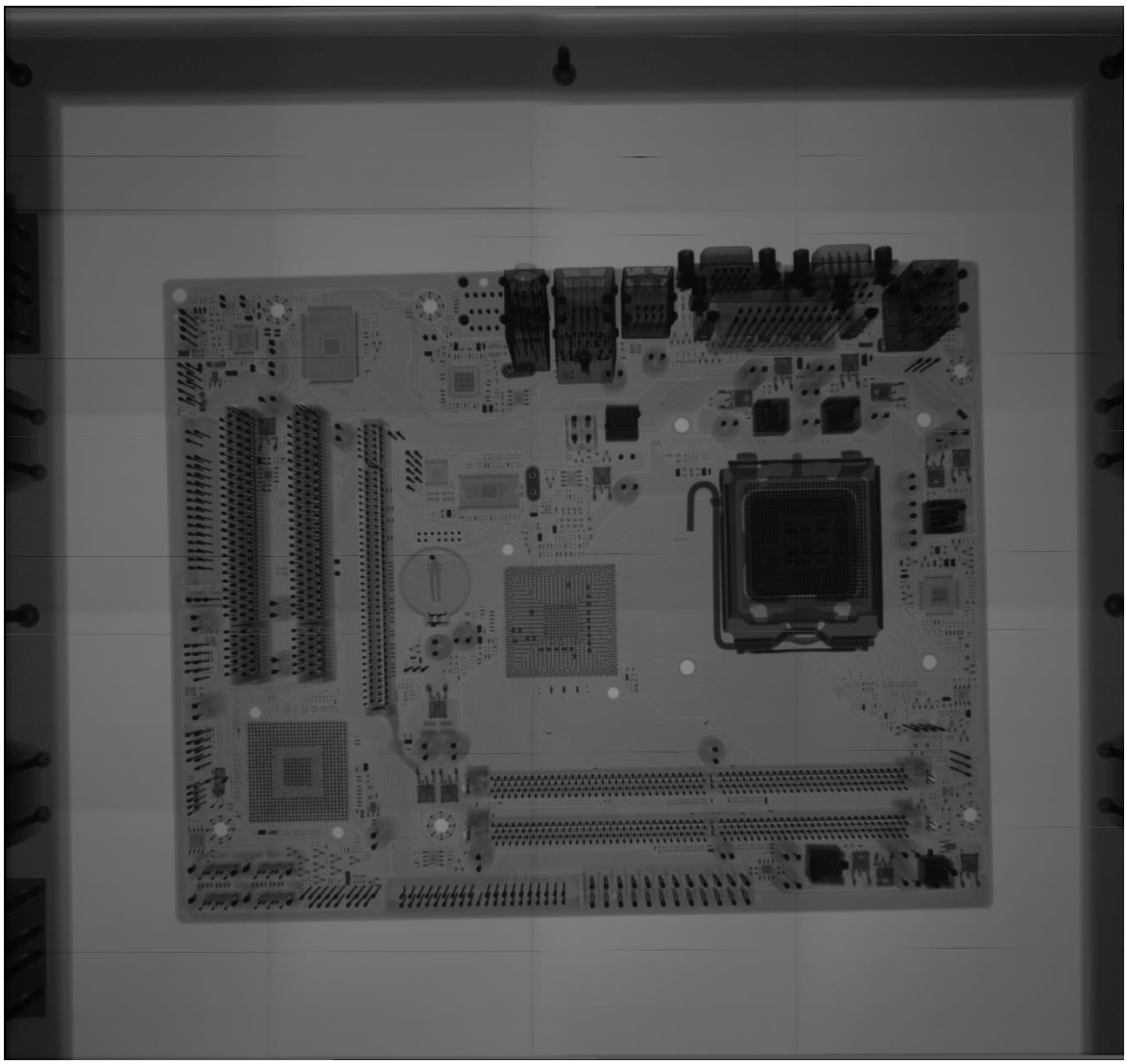


Рисунок 3.5 – Сшитое рентгеновское изображение, исследуемой печатной платы, для первого комплекта изображений

Аналогичные результаты получены и для остальных комплектов изображений, они представлены на рисунках 3.6, 3.7 и 3.8 соответственно.

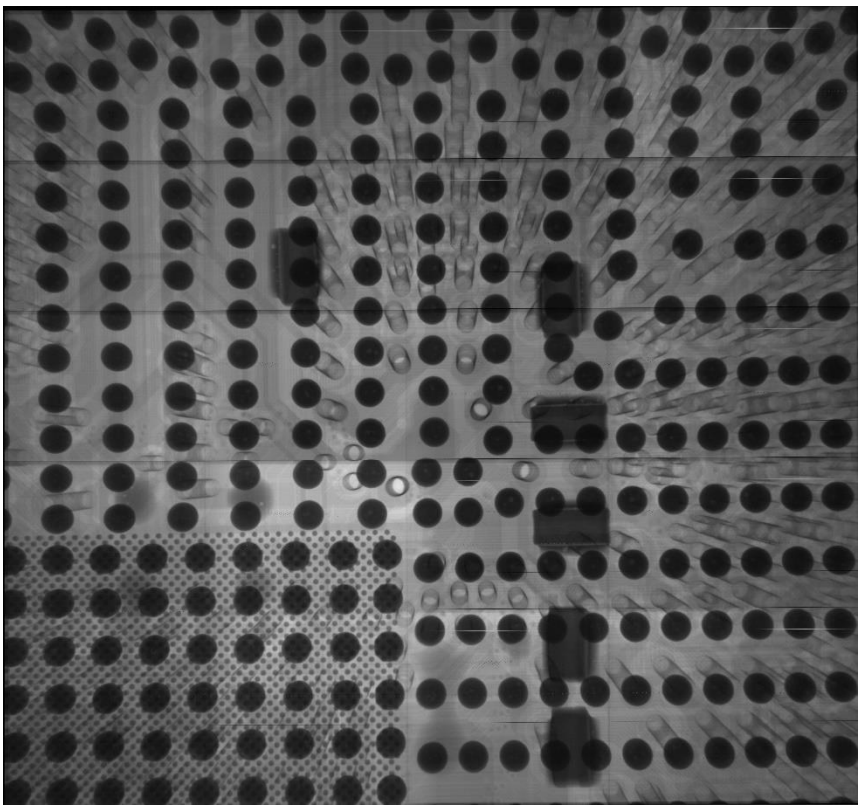


Рисунок 3.6 – Сшитое рентгеновское изображение, исследуемой печатной платы, для второго комплекта изображений

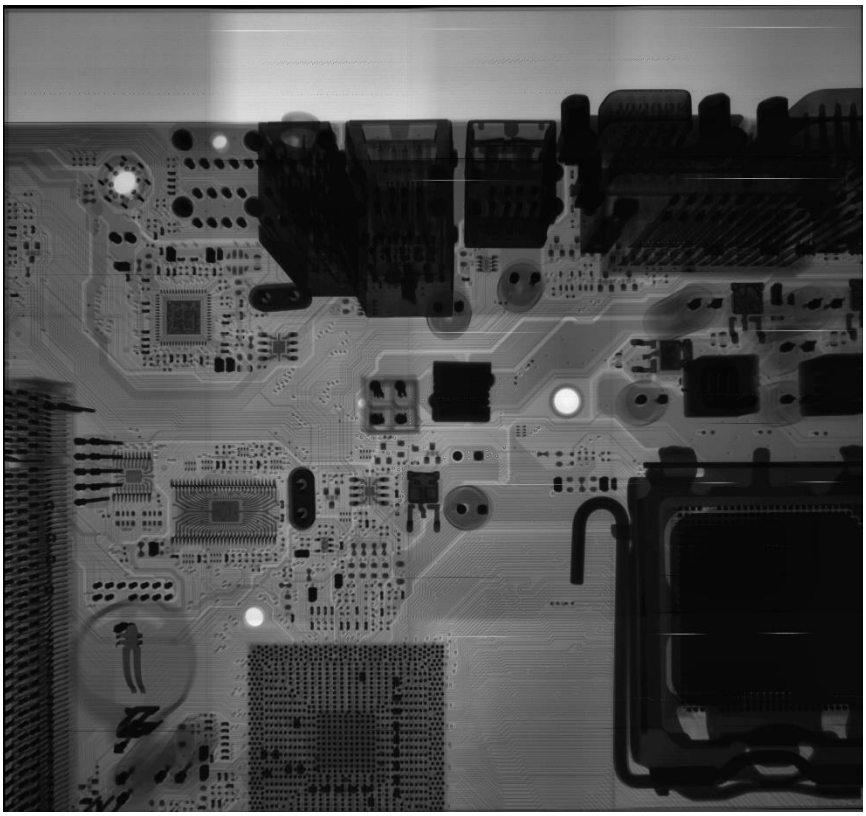


Рисунок 3.7 – Сшитое рентгеновское изображение, исследуемой печатной платы, для третьего комплекта изображений

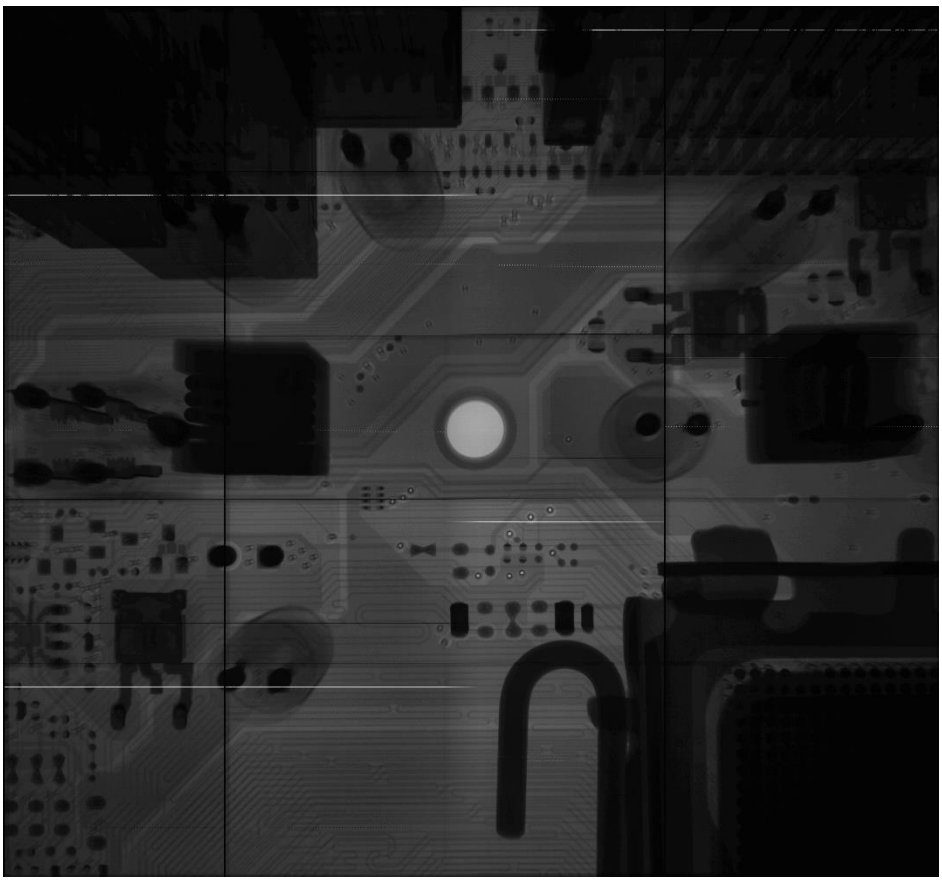


Рисунок 3.8 – Сшитое рентгеновское изображение, исследуемой печатной платы, для четвертого комплекта изображений

ЗАКЛЮЧЕНИЕ

В рамках выполненной работы была разработана система для автоматизированной сшивки рентгеновских изображений, основанная на алгоритмах выделения и сопоставления ключевых точек. Реализация предложенного подхода позволила компенсировать геометрические искажения, возникающие при получении изображений, и обеспечить точное наложение кадров. Проведено тестирование алгоритма на реальных данных, полученных с использованием рентгенографической системы неразрушающего контроля …, что подтвердило его эффективность для обработки сложных структур, таких как печатные платы.

Полученные результаты продемонстрировали возможность точного совмещения изображений, а также выявили проблемы, связанные с неоднородностью яркости и засветкой в исходных данных. Для повышения скорости работы алгоритма предложено перевести сложные вычисления с процессора на видеопамять, что ускорило бы процесс вычисления ключевых точек. Также рассматривается возможность использования современных нейросетей для задачи вычисления и сопоставления ключевых точек на изображениях.

Разработанный алгоритм и проведенные эксперименты показывают перспективность подхода для задач неразрушающего контроля и создания комплексных решений в области рентгеновской диагностики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Siddique Abu Bakar , Xiaoming Jiang , Xiangfu Gui , Guoquan Li and Zhangyong Li; «Image Stitching for Chest Digital Radiography Using the SIFT and SURF Feature Extraction by RANSAC Algorithm», 2020 – 7 p. doi:10.1088/1742-6596/1624/4/042023

2. С.В. Шаветов, А.Д. Жданов. Основы обработки изображений: лабораторный практикум. — СПб: Университет ИТМО, 2022. — 122 с.

3. Fan Yang, Yan He, Zhen Sheng Deng & Ang Yan; “Improvement of automated image stitching system for DR X-ray images”, Computers in Biology and Medicine (2016) 71:108–114.

4. Paudel Krishna; Uppsala University, Disciplinary Domain of Science and Technology, Mathematics and Computer Science, Department of Information Technology; « Stitching of X-ray Images» (2012) 78 p.

5. Кэлер А., Брэдски Г. Изучаем OpenCV 3 / пер. с анг. А. А. Слинкина. – М.: ДМК Пресс, 2017. – 826 с.: ил.

ПРИЛОЖЕНИЕ А
ЛИСТИНГ АЛГОРИТМА АВТОМАТИЧЕСКОЙ СШИВКИ ИЗОБРАЖЕНИЙ

```
#include "opencv2/opencv.hpp"
#include <opencv2/features2d.hpp>
#include <opencv2/highgui.hpp>
#include <opencv2/imgproc.hpp>
#include <opencv2/calib3d.hpp>
#include <chrono>

int main ()
{
    setlocale (0, "Russian");
    int n = 20;
    int k = 0;
    std::string name_image2;
    std::string name_image3;
    std::string name_image1;
    cv:: Mat img2;
    cv:: Mat img1;
    std::vector<cv::KeyPoint> kpoints;
    cv::Mat dpoints;

    auto start = std::chrono::high_resolution_clock::now();
    cv::Ptr<cv::CLAHE> clahe = cv::createCLAHE(2.0, cv::Size(4, 4));

    for (int i = 1; i < n; i++) {
        if(i%5 == 0){
            k++;
            continue;
        }
        name_image1 = "testing\\wholeImage_" + std::to_string(i+1)+
"_20.tiff";
        img1 = cv::imread(name_image1, cv:: IMREAD_GRAYSCALE);

        std:: cout << "i: "<< i << std:: endl;
        std:: cout << "k: "<< k << std:: endl;
        name_image3 = "stitching_image" + std::to_string(k)+ ".tiff";
        if(i%5 == 1) {

            name_image2 = "testing\\wholeImage_" + std::to_string(i)+
"_20.tiff";
            img2 = cv::imread(name_image2, cv:: IMREAD_GRAYSCALE);
            cv::normalize(img2, img2, 0, 255, cv::NORM_MINMAX);
            clahe->apply(img2, img2);


        }
        else{
            img2 = cv::imread(name_image3, cv:: IMREAD_GRAYSCALE);
        }
```

```
        cv::normalize(img1, img1, 0, 255, cv::NORM_MINMAX);
        clahe->apply(img1, img1);
        img1.convertTo(img1, CV_8U);
        img2.convertTo(img2, CV_8U);



        if (img1.empty() || img2.empty()){
            std::cerr << "image read error";
            return -1;
        }

        std::vector<cv::KeyPoint> keypoints1, keypoints2;
        cv:: Mat descriptors1, descriptors2;
        std:: cout << "checkpointSIFT1" << std::endl;

        cv::Mat mask_roi = cv::Mat::zeros(img2.size(), CV_8U);
        int rect_x = 0;
        int rect_y = 0;
        int rect_width = img2.cols;
        int rect_height = img1.rows;
        cv::Rect roi(rect_x, rect_y, rect_width,
rect_height);
        cv::rectangle(mask_roi, roi, cv::Scalar(255), cv::FILLED);
        cv::imwrite("mask1.png", mask_roi);

        cv::Ptr<cv::SIFT> sift= cv::SIFT::create(0, 5);
        sift->detectAndCompute(img1, cv::Mat(), keypoints1,
descriptors1);
        sift->detectAndCompute(img2, cv::Mat(), keypoints2,
descriptors2);

        cv::FlannBasedMatcher matcher(new cv::flann::KDTreeIndexParams(),
new cv::flann::SearchParams());
        descriptors1.convertTo(descriptors1, CV_32F);
        descriptors2.convertTo(descriptors2, CV_32F);
        std::vector<std::vector<cv::DMatch>> knn_matches;
        matcher.knnMatch(descriptors1, descriptors2, knn_matches, 2);

        std:: vector<cv::DMatch> good_matches;


        const float ratio_thresh = 0.65f;
        for (const auto& match : knn_matches) {
            if (match[0].distance < ratio_thresh * match[1].distance) {
               good_matches.push_back(match[0]);
            }
        }

        if (good_matches.size() < 20) {

                std::cout << "SIFT no matches" << std::endl;
                cv::Mat res;
                cv::Mat croppedImg = img1(cv::Rect(0, 0, img1.cols,
img1.rows - 540));

                if (img1.empty() || img2.empty()) {
```

```
                    std::cerr << "Error: One or more images could not be
loaded!" << std::endl;
                    return -1;
                }

                // Проверка размеров и типов
                if (img1.cols != img2.cols) {
                    std::cout << "weight:" << img1.cols << " 1 2 " <<
img2.cols << std::endl;
                    std::cerr << "Error: Image widths do not match!" <<
std::endl;
                    return -1;
                }
                if (img1.type() != img2.type()) {
                    std::cerr << "Error: Image types do not match!" <<
std::endl;
                    return -1;
                }

                cv::vconcat(croppedImg, img2, res);
                cv::imwrite(name_image3, res);
                continue;
        }



        std::vector<cv::Point2f> points1, points2;
        for (size_t i =0; i < good_matches.size(); i++){
            points1.push_back(keypoints1[good_matches[i].queryIdx].pt);
            points2.push_back(keypoints2[good_matches[i].trainIdx].pt);
        }

        cv::Mat affine_matrix = cv::estimateAffinePartial2D(points2,
points1);
        std::vector<cv::Point2f> img2_corners(4);
        img2_corners[0] = cv::Point2f(0,0);
        img2_corners[1] = cv::Point2f(static_cast<float>(img2.cols), 0);
        img2_corners[2] = cv::Point2f(0, static_cast<float>(img2.rows));
        img2_corners[3] = cv::Point2f(static_cast<float>(img2.cols),
static_cast<float>(img2.rows));
        std::vector<cv::Point2f> img2_transformed_corners(4);

        cv::transform(img2_corners, img2_transformed_corners,
affine_matrix);

        float min_x = std::min(0.0f,
std::min({img2_transformed_corners[0].x, img2_transformed_corners[1].x,
img2_transformed_corners[2].x }));
        float min_y = std::min(0.0f,
std::min({img2_transformed_corners[0].y, img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));
        float max_x = std::max(static_cast<float>(img1.cols),
std::max({img2_transformed_corners[0].x, img2_transformed_corners[1].x,
img2_transformed_corners[2].x}));
        float max_y = std::max(static_cast<float>(img1.rows),
std::max({img2_transformed_corners[0].y, img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));
```

```
        cv::Size result_size(img1.cols, static_cast<int>(std::ceil(max_y
- min_y)));

        cv::Mat result(result_size, img1.type(), cv::Scalar::all(255));
        cv::Mat translation_matrix = (cv::Mat_<double>(2, 3) << 1, 0, -
min_x, 0 , 1, -min_y);
        cv::warpAffine(img1, result, translation_matrix, result_size,
cv::INTER_LINEAR, cv::BORDER_REPLICATE);

        cv::Mat img2_transformed;
        cv::Mat img2_shifted;
        cv::warpAffine(img2, img2_shifted, translation_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REPLICATE);
        cv::warpAffine(img2_shifted, img2_transformed, affine_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REPLICATE);


        cv::Mat mask = cv::Mat::zeros(result.size(), CV_32F);
        int blendWidth = 100;
        cv::Rect
blendRect(static_cast<int>(img2_transformed_corners[0].x),
                  static_cast<int>(img2_transformed_corners[0].y),
                  img2.cols + blendWidth, img2.rows + blendWidth);

        blendRect.x = std::max(blendRect.x, 0);
        blendRect.y = std::max(blendRect.y, 0);
        blendRect.width = std::min(blendRect.width + blendWidth,
mask.cols - blendRect.x);
        blendRect.height = std::min(blendRect.height + blendWidth,
mask.rows - blendRect.y);

        for (int y = blendRect.y; y < blendRect.y + blendRect.height &&
< mask.rows; ++y) {
            for (int x = blendRect.x; x < blendRect.x + blendRect.width
&& x < mask.cols; ++x) {
                float alpha = static_cast<float>(y - blendRect.y) /
blendWidth;
                alpha = std::min(std::max(alpha, 0.0f), 1.0f);

                    mask.at<float>(y, x) = alpha;


            }
        }

        result.convertTo(result, CV_32F);
        img2_transformed.convertTo(img2_transformed, CV_32F);



        cv::Mat blended;
        blended = result.mul(1.0 - mask) + img2_transformed.mul(mask);
        blended.convertTo(blended, CV_16F);
        cv::imwrite(name_image3, blended);
        blended.convertTo(blended, CV_8U);
```

```
        img1.release();
        img2.release();
        affine_matrix.release();
        translation_matrix.release();
        result.release();
        img2_shifted.release();
        img2_transformed.release();
        mask.release();
        blended.release();

        points1.clear();
        points2.clear();
        descriptors1.release();
        descriptors2.release();
        keypoints1.clear();
        keypoints2.clear();
        good_matches.clear();

    }




    //Combining vertical stripes
    //Horizontal stitching

     for (int i = 0; i < 3; i++) {

        if (i == 0){
            name_image1 = "stitching_image" + std::to_string(i)+ ".tiff";
        }
        else {
            name_image1 = "final_image.tiff";
        }
        name_image2 = "stitching_image" + std::to_string(i+1)+ ".tiff";

        img2 = cv::imread(name_image2, cv:: IMREAD_GRAYSCALE);

        std:: cout << "i: "<< i << std:: endl;

        img1= cv::imread(name_image1, cv:: IMREAD_GRAYSCALE);
        img1.convertTo(img1, CV_8U);
        img2.convertTo(img2, CV_8U);

        std::vector<cv::KeyPoint> keypoints1, keypoints2;
        cv:: Mat descriptors1, descriptors2;

        cv::Mat mask_roi = cv::Mat::zeros(img1.size(), CV_8U);
        int rect_x = std::max(0, img1.cols - img2.cols);
        int rect_y = 0;
        int rect_width = std::min(img2.cols, img1.cols);
        int rect_height = img1.rows;
        cv::Rect roi(rect_x, rect_y, rect_width,
rect_height);
        cv::rectangle(mask_roi, roi, cv::Scalar(255), cv::FILLED);
        cv::imwrite("mask.png", mask_roi);
```

```
        cv::Ptr<cv::SIFT> sift = cv::SIFT::create();

        sift->detectAndCompute(img1, mask_roi, keypoints1, descriptors1);

        sift->detectAndCompute(img2, cv::Mat(), keypoints2,
descriptors2);



        cv::FlannBasedMatcher matcher(new cv::flann::KDTreeIndexParams(),
new cv::flann::SearchParams());
        descriptors1.convertTo(descriptors1, CV_32F);
        descriptors2.convertTo(descriptors2, CV_32F);
        std::vector<std::vector<cv::DMatch>> knn_matches;
        matcher.knnMatch(descriptors1, descriptors2, knn_matches, 2);



        const float ratio_thresh = 0.65f;
        std:: vector<cv::DMatch> good_matches;

        for (const auto& match : knn_matches) {
            if (match[0].distance < ratio_thresh * match[1].distance) {
                good_matches.push_back(match[0]);
            }
        }



        if (good_matches.size() < 10) {


                cv::Mat res;
                cv::Mat croppedImg = img1(cv::Rect(0, 0, img1.cols-580,
img1.rows));

                if (img1.empty() || img2.empty()) {
                    std::cerr << "Error: One or more images could not be
loaded!" << std::endl;
                    return -1;
                }

                // Проверка размеров и типов
                if (img1.rows != img2.rows) {
                    std::cout << "weight:" << img1.rows << " 1 2 " <<
img2.rows << std::endl;
                    std::cerr << "Error: Image widths do not match!" <<
std::endl;
                    return -1;
                }
                if (img1.type() != img2.type()) {
                    std::cerr << "Error: Image types do not match!" <<
std::endl;
                    return -1;
                }
```

```
                cv::hconcat(croppedImg, img2, res);
                cv::imwrite("final_image.tiff", res);
                continue;
        }



        std::vector<cv::Point2f> points1, points2;

        for (size_t i =0; i < good_matches.size(); i++){

            if (good_matches[i].queryIdx >= keypoints1.size() ||
good_matches[i].trainIdx >= keypoints2.size()) {
            std::cerr << "Error: Index out of range at match " << i <<
std::endl;
            }

            points1.push_back(keypoints1[good_matches[i].queryIdx].pt);
            points2.push_back(keypoints2[good_matches[i].trainIdx].pt);
        }

        cv::Mat affine_matrix = cv::estimateAffinePartial2D(points2,
points1);

        std::vector<cv::Point2f> img2_corners(4);

        img2_corners[0] = cv::Point2f(0,0);
        img2_corners[1] = cv::Point2f(static_cast<float>(img2.cols), 0);
        img2_corners[2] = cv::Point2f(0, static_cast<float>(img2.rows));
        img2_corners[3] = cv::Point2f(static_cast<float>(img2.cols),
static_cast<float>(img2.rows));
        std::vector<cv::Point2f> img2_transformed_corners(4);


        cv::transform(img2_corners, img2_transformed_corners,
affine_matrix);



        float min_x = std::min(0.0f,
std::min({img2_transformed_corners[0].x, img2_transformed_corners[1].x,
img2_transformed_corners[2].x }));
        float min_y = std::min(0.0f,
std::min({img2_transformed_corners[0].y, img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));
        float max_x = std::max(static_cast<float>(img1.cols),
std::max({img2_transformed_corners[0].x, img2_transformed_corners[1].x,
img2_transformed_corners[2].x}));
        float max_y = std::max(static_cast<float>(img1.rows),
std::max({img2_transformed_corners[0].y, img2_transformed_corners[1].y,
img2_transformed_corners[2].y}));


        cv::Size result_size(static_cast<int>(std::ceil(max_x - min_x)),
img1.rows);
        cv::Mat result(result_size, img1.type(), cv::Scalar::all(255));
```

```
        cv:: Mat translation_matrix = (cv::Mat_<double>(2, 3) << 1, 0, -
min_x, 0 , 1, -min_y);
        cv::warpAffine(img1, result, translation_matrix, result_size,
cv::INTER_LINEAR, cv::BORDER_REPLICATE);


        cv::Mat img2_shifted;
        cv::warpAffine(img2, img2_shifted, translation_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REPLICATE);

        cv::Mat img2_transformed;
        cv::warpAffine(img2_shifted, img2_transformed, affine_matrix,
result_size, cv::INTER_LINEAR, cv::BORDER_REPLICATE);

        cv::Mat mask = cv::Mat::zeros(result.size(), CV_32F);
        int blendWidth = 400;

        cv::Rect
blendRect(static_cast<int>(img2_transformed_corners[0].x),
                    static_cast<int>(img2_transformed_corners[0].y),
                    img2.cols + blendWidth, img2.rows);

        blendRect.x = std::max(blendRect.x, 0);
        blendRect.y = std::max(blendRect.y, 0);
        blendRect.width = std::min(blendRect.width, mask.cols -
blendRect.x);
        blendRect.height = std::min(blendRect.height, mask.rows -
blendRect.y);

        for (int y = blendRect.y; y < blendRect.y + blendRect.height && y
< mask.rows; ++y) {
            for (int x = blendRect.x; x < blendRect.x + blendRect.width
&& x < mask.cols; ++x) {
                float alpha = static_cast<float>(x - blendRect.x) /
blendWidth;
                alpha = std::min(std::max(alpha, 0.0f), 1.0f);

                    mask.at<float>(y, x) = alpha;


            }
        }
        result.convertTo(result, CV_32F);
        img2_transformed.convertTo(img2_transformed, CV_32F);
        cv::Mat blended;

        blended = result.mul(1.0 - mask) + img2_transformed.mul(mask);

        blended.convertTo(blended, CV_16F);

        cv::imwrite("final_image.tiff", blended);

        img1.release();
        img2.release();

        affine_matrix.release();
        translation_matrix.release();
```

```
        result.release();
        img2_shifted.release();
        img2_transformed.release();
        mask.release();
        blended.release();



        points1.clear();
        points2.clear();

        descriptors1.release();
        descriptors2.release();
        keypoints1.clear();
        keypoints2.clear();
        good_matches.clear();
        knn_matches.clear();
    }

    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> elapsed = end - start;
    std::cout << "times: " << elapsed.count() << " s." << std::endl;
    return 0;
}
```